Code

Stats 20, F23 – Homework 2

Aashna Sibal

10/23/2023

Question 1

(a)

```
commute_times <- c(14, 12, 20, 19, 15, 20, 28, 20, 20, 18)

which(abs((commute_times - mean(commute_times))) > sd(commute_times))
```

```
## [1] 1 2 7
```

```
commute_times[c(1,2,7)]
```

```
## [1] 14 12 28
```

On day 1, 2, and 7 Andy had a commute time more than one SD from the mean commute time. Those commute times were 14, 12, and 28 minutes.

(b)

```
which(abs((commute_times - mean(commute_times))) <= sd(commute_times))
```

```
## [1]  3  4  5  6  8  9 10
```

```
commute_times[c(3,4,5,6,8,9,10)]
```

```
## [1] 20 19 15 20 20 20 18
```

Andy had a commute time within one SD from the mean on days 3, 4, 5, 6, 8, 9, and 10. These commute times are shown above in minutes–20,19,15,20,20,20,18.

(c)

```
#sum(abs((commute_times - mean(commute_times))) <= sd(commute_times))/
length(commute_times)

sum(!(abs((commute_times - mean(commute_times))) >
sd(commute_times)))/length(commute_times)
```

```
## [1] 0.7
```

70% of the time, he had a commute time that was within one standard deviation of the mean commute time.

Question 2

```
NA & TRUE
```

```
## [1] NA
```

```
NA & FALSE
```

```
## [1] FALSE
```

```
NA | TRUE
```

```
## [1] TRUE
```

```
NA | FALSE
```

```
## [1] NA
```

When there is an and, both conditions must be true in order for the function to be true– however, R does not know what NA is, it could be true or false. Therefore, it does not know if it is true or false statement. The second one is false since as long as there is one false, it remains false, and will not be true–it does not matter what NA is. When there is an or, you just need TRUE on either side, and since that is present in line 3, it outputs true. For the fourth one, NA could be true, so since it is a possibility, the output is unknown.

Question 3

```
get_minimum_coins <- function(cents) {
  q <- cents %/% 25
  d <- (cents - (25 * q)) %/% 10
  n <- (cents - (25 * q) - (10 * d)) %/% 5
  p <- (cents - (25 * q) - (10 * d) - (5 * n)) %/% 1

  q + d + n + p
}
```

(a)

Yes, this is a vectorized function because when I try it on a vector (below) it applies the function to each element in the vector one by one.

```
cents <- 1:10
get_minimum_coins(cents)
```

```
##  [1] 1 2 3 4 1 2 3 4 5 1
```

(b)

```
which(get_minimum_coins(1:100) == max(get_minimum_coins(1:100)))
```

```
## [1] 94 99
```

Question 4

(a)

```
my_min <- function(x, na.rm = FALSE){
  if (na.rm){
    x <- x[!is.na(x)]
  }

  min <- x[1]

    for (n in x) {
      if (is.na(n)) {
        min <- NA
        break
      }
      if (n <= min) {
        min <- n
      }

    }
  min
  }
```

(b)

```
my_min(c(4, 1, 0, 2, -3, -5, -4))
```

```
## [1] -5
```

```
my_min(c("bears", "beets", "Battlestar Galactica"))
```

```
## [1] "Battlestar Galactica"
```

```
my_min(7)
```

```
## [1] 7
```

```
my_min(c("Pawnee", "rules", "Eagleton", NA), na.rm = TRUE)
```

```
## [1] "Eagleton"
```

```
my_min(c("Pawnee", "rules", "Eagleton", NA))
```

```
## [1] NA
```

```
my_min(NA, na.rm = TRUE)
```

```
## [1] NA
```

```
my_min(NA, na.rm = FALSE)
```

```
## [1] NA
```

Question 5

(a)

```
fib1 <- 1
fib2 <- 1
full_fib <- c(fib1, fib2)
while (fib1 + fib2 < 500) {
  fib2 <- fib1 + fib2
  full_fib <- c(full_fib, fib2)
  fib1 <- fib2 - fib1
}
full_fib
```

```
##  [1]   1   1   2   3   5   8  13  21  34  55  89 144 233 377
```

(b)

```
i <- 1
full_fib <- c(1,1)
while (full_fib[i] + full_fib[i+1] < 500) {
  for (i in length(full_fib)){
    full_fib[i+1] <- full_fib[i] + full_fib[i-1]
    }
}
full_fib
```

```
##  [1]   1   1   2   3   5   8  13  21  34  55  89 144 233 377
```

(c)

```
i <- 1
full_fib <- c(1,1)
while (full_fib[i] + full_fib[i+1] < 10^9) {
  for (i in length(full_fib)){
    full_fib[i+1] <- full_fib[i] + full_fib[i-1]
    }
}
length(full_fib)
```

```
## [1] 44
```

There are 44 Fibonacci numbers less than 1 billion.

Question 6

```
jerry <- 2:(8 * 5 %% 3^-2:(7 > 2))
jerry
```

```
## [1] 2 1 0
```

Question 7

(a)

```
my_ifelse <- function(test,yes,no) {
  output <- 0
  output[which(test)] <- yes[which(test)]
  output[which(!test)] <- no[which(!test)]
  output
}
```

(b)

```
x <- (1:10) * pi
my_ifelse(x %% 1 >= 0.5, x %/% 1 + 1, x %/% 1)
```

```
##  [1]  3  6  9 13 16 19 22 25 28 31
```

(c)

```
my_abs <- function(x) {
  my_ifelse(x >= 0, x, -x)
}

my_sign <- function(x) {
  result <- my_ifelse(x > 0, x/x, -x/x)
  result[which(is.nan(result))] <- 0
  result
}
```

Question 8

(a)

```
merge <- function(left,right) {
  merged <- 0
  while (length(left) > 0) {
    if (length(right) > 0) {
      if (left[1] < right[1]) {
        merged <- c(merged,left[i])
        left <- left[-1]
      } else {
        merged <- c(merged,right[i])
        right <- right[-1]}
    } else {
      merged <- c(merged,left)
      left <- 0}
  }
  c(merged,right)
}

merge_sort <- function(x) {
  if (length(x) > 1) {
    half <- length(x)/2
    left <- x[1:half]
    right <- x[half+1:length(x)]
    sorted_left <- merge_sort(left)
    sorted_right <- merge_sort(right)
    x <- merge(sorted_left, sorted_right)
  }
  x
}
```

(b)

```
merge_sort(numeric(0))
```

```
## numeric(0)
```

```
merge_sort(7)
```

```
## [1] 7
```

```
#merge_sort(10:1)
#commented out due to error "Error: C stack usage  7955000 is too
close to the limit"
```

Question 9

```
load("dna.RData")
```

(a)

(b)